Lab7 SVM

Source: MLL/Lab7 SVM.ipynb

CELL 1 — CODE

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
accuracy_score,
precision_score,
recall_score,
f1_score,
classification_report,
confusion_matrix
)
```

CELL 2 — CODE

```
iris = load_iris()
print("Dataset loaded successfully")
print("Number of samples:", iris.data.shape[0])
print("Number of features:", iris.data.shape[1])
print("Classes:", iris.target_names)
```

OUTPUT

```
Dataset loaded successfully
Number of samples: 150
Number of features: 4
Classes: ['setosa' 'versicolor' 'virginica']
```

CELL 3 — CODE

```
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["Target"] = iris.target
df["Species"] = df["Target"].map(dict(enumerate(iris.target_names)))
display(df.head())
print("\nDataset Shape:", df.shape)
```

OUTPUT

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   Target Species
0       0  setosa
1       0  setosa
2       0  setosa
3       0  setosa
4       0  setosa

Dataset Shape: (150, 6)
```

CELL 4 — CODE

```
X = iris.data
y = iris.target
print("Features (X):")
print(X[:5])
print("\nTarget (y):")
print(y[:5])
```

OUTPUT

```
Features (X):
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]

Target (y):
[0 0 0 0 0]
```

CELL 5 — CODE

```
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.30,
random_state=42,
stratify=y
)
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

OUTPUT

```
Training samples: 105
Testing samples: 45
```

CELL 6 — CODE

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("Feature scaling completed.")
```

OUTPUT

```
Feature scaling completed.
```

CELL 7 — CODE

(Empty cell)

CELL 8 — CODE

```
linear_svm = SVC(kernel="linear")
linear_svm.fit(X_train_scaled, y_train)
linear_pred = linear_svm.predict(X_test_scaled)
print("Linear SVM training completed.")
```

OUTPUT

```
Linear SVM training completed.
```

CELL 9 — CODE

```
linear_accuracy = accuracy_score(y_test, linear_pred)
linear_precision = precision_score(y_test, linear_pred, average="weighted")
linear_recall = recall_score(y_test, linear_pred, average="weighted")
linear_f1 = f1_score(y_test, linear_pred, average="weighted")
print("LINEAR SVM PERFORMANCE")
print("----------------------")
print("Accuracy :", linear_accuracy)
print("Precision:", linear_precision)
print("Recall :", linear_recall)
print("F1 Score :", linear_f1)
print("\nClassification Report:")
print(classification_report(
y_test,
linear_pred,
target_names=iris.target_names
))
```

OUTPUT

```
LINEAR SVM PERFORMANCE
----------------------
Accuracy : 0.9111111111111111
Precision: 0.9155354449472096
Recall : 0.9111111111111111
F1 Score : 0.9107142857142857

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.82      0.93      0.88        15
   virginica       0.92      0.80      0.86        15

    accuracy                           0.91        45
   macro avg       0.92      0.91      0.91        45
weighted avg       0.92      0.91      0.91        45
```

CELL 10 — CODE

```
print("Linear SVM Confusion Matrix:")
print(confusion_matrix(y_test, linear_pred))
```

OUTPUT

```
Linear SVM Confusion Matrix:
[[15  0  0]
 [ 0 14  1]
 [ 0  3 12]]
```

CELL 11 — CODE

```
rbf_svm = SVC(kernel="rbf")
rbf_svm.fit(X_train_scaled, y_train)
rbf_pred = rbf_svm.predict(X_test_scaled)
print("RBF SVM training completed.")
```

OUTPUT

```
RBF SVM training completed.
```

CELL 12 — CODE

```
rbf_accuracy = accuracy_score(y_test, rbf_pred)
rbf_precision = precision_score(y_test, rbf_pred, average="weighted")
rbf_recall = recall_score(y_test, rbf_pred, average="weighted")
rbf_f1 = f1_score(y_test, rbf_pred, average="weighted")
print("RBF SVM PERFORMANCE")
print("--------------------")
print("Accuracy :", rbf_accuracy)
print("Precision:", rbf_precision)
print("Recall :", rbf_recall)
print("F1 Score :", rbf_f1)
print("\nClassification Report:")
print(classification_report(
y_test,
rbf_pred,
target_names=iris.target_names
))
```

OUTPUT

```
RBF SVM PERFORMANCE
--------------------
Accuracy : 0.9333333333333333
Precision: 0.9345238095238095
Recall : 0.9333333333333333
F1 Score : 0.9332591768631814

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.88      0.93      0.90        15
   virginica       0.93      0.87      0.90        15

    accuracy                           0.93        45
   macro avg       0.93      0.93      0.93        45
weighted avg       0.93      0.93      0.93        45
```

CELL 13 — CODE

```
print("RBF SVM Confusion Matrix:")
print(confusion_matrix(y_test, rbf_pred))
```

OUTPUT

```
RBF SVM Confusion Matrix:
[[15  0  0]
 [ 0 14  1]
 [ 0  2 13]]
```

CELL 14 — CODE

```
comparison = pd.DataFrame({
"Classifier": ["Linear SVM", "RBF SVM"],
"Accuracy": [linear_accuracy, rbf_accuracy],
"Precision": [linear_precision, rbf_precision],
"Recall": [linear_recall, rbf_recall],
"F1 Score": [linear_f1, rbf_f1]

})
```

CELL 15 — CODE

```
comparison_percentage = comparison.copy()
metrics = ["Accuracy", "Precision", "Recall", "F1 Score"]
for metric in metrics:
  comparison_percentage[metric] = (
    comparison_percentage[metric] * 100
).round(2)
print("SVM Classification Performance (%)")
display(comparison_percentage)
```

OUTPUT

```
SVM Classification Performance (%)

   Classifier  Accuracy  Precision  Recall  F1 Score
0  Linear SVM     91.11      91.55   91.11     91.07
1     RBF SVM     93.33      93.45   93.33     93.33
```

CELL 16 — CODE

```
best_model = comparison.loc[comparison["F1 Score"].idxmax()]
print("Best Performing Classifier:")
print(best_model["Classifier"])
print("\nBest F1 Score:", round(best_model["F1 Score"], 4))
print("Best Accuracy:", round(best_model["Accuracy"], 4))
```

OUTPUT

```
Best Performing Classifier:
RBF SVM

Best F1 Score: 0.9333
Best Accuracy: 0.9333
```

CELL 17 — CODE

```
print("========== LINEAR SVM ==========")
print(confusion_matrix(y_test, linear_pred))
print("\n========== RBF SVM ==========")
print(confusion_matrix(y_test, rbf_pred))
```

OUTPUT

```
========== LINEAR SVM ==========
[[15  0  0]
 [ 0 14  1]
 [ 0  3 12]]

========== RBF SVM ==========
[[15  0  0]
 [ 0 14  1]
 [ 0  2 13]]
```

CELL 18 — CODE

```
if rbf_f1 > linear_f1:
  print("Conclusion: RBF SVM performed better than Linear SVM based on the F1 Score.")
elif linear_f1 > rbf_f1:
  print("Conclusion: Linear SVM performed better than RBF SVM based on the F1 Score.")
else:
  print("Conclusion: Both SVM classifiers achieved the same F1 Score.")
```

OUTPUT

```
Conclusion: RBF SVM performed better than Linear SVM based on the F1 Score.
```